



南开大学
Nankai University

《软件体系结构》课程作业报告

LabScribe Agent 体系结构设计文档

课程名称：软件体系结构

学 院：软件学院

姓 名：薛以贤 李宇帆

学 号：2312386 2312391

指导老师：张重群

2026 年 7 月 4 日

目录

1 项目总览	1
1.1 项目概述	1
1.2 课程设计目标	1
2 用户需求分析	2
2.1 应用场景与目标用户	2
2.2 功能性需求	2
2.3 非功能性需求	4
2.4 核心交互流程	4
3 功能分解与总体结构	5
3.1 顶层功能分解	5
3.2 主要模块职责	6
3.3 代码组织结构	7
3.4 核心架构约束	8
4 软件体系结构风格设计	8
4.1 三层 B/S 风格	9
4.2 仓储风格	9
4.3 管道—过滤器风格	10
4.4 面向对象风格	10
4.5 风格选择总结	10
5 智能体设计模式分析	11
5.1 RAG：检索增强生成模式	11
5.2 Tool Use：工具使用模式	12
5.3 Prompt Chaining 与 Planner-Executor 模式	12
5.4 Routing：路由模式	13
5.5 Memory Management：记忆管理模式	13
5.6 Guardrails 与异常恢复模式	13
6 4+1 视图分析	14
6.1 逻辑视图：核心类与组件结构	14
6.2 开发视图：模块划分与代码组织	15

6.3 进程视图：并发、调度与通信机制	17
6.4 物理视图：部署结构与软硬件映射	18
6.5 场景视图：关键用例与流程串联	19
6.6 五个视图之间的一致性	20
7 数据设计与接口设计	21
7.1 数据模型设计	21
7.2 API 接口设计	22
7.3 RAG 检索算法	22
8 关键实现机制	23
9 体系结构质量评估	23
9.1 质量属性场景	23
9.2 敏感点与权衡点	24
10 Demo 实现与运行说明	24
10.1 实验环境	24
10.2 安装与启动	24
10.3 演示流程设计	25
11 结论	25
附录：个人分工表	27

1 项目总览

实验报告编写需要综合处理课程模板、实验材料、运行截图和结果数据。通用大模型难以同时保证内容可追溯、模板结构稳定和图片引用准确，因此我们设计并实现了面向高校实验场景的 **LabScribe Agent 实验报告编写助手**。

我们将系统建模为由前端、FastAPI 接口、报告智能体、文档解析、OCR、RAG、LLM、LaTeX 导出和 SQLite 仓储等构件组成的软件体系结构，并通过 HTTP、函数调用、数据库访问和任务轮询连接各构件。报告重点分析需求、功能分解、体系结构风格、智能体模式和 4+1 视图，同时说明实现、评估与验证结果。

1.1 项目概述

我们实现的 LabScribe Agent 是一个三层 B/S Web 应用。它可读取 Word、LaTeX、PDF、TXT、Markdown 和实验截图，使用 OCR 提取图片文字，以 Ollama 或本地哈希生成向量，并在 SQLite 中维护轻量 RAG 知识库。系统调用 DeepSeek 完成问答与报告规划，最终导出可继续编译和编辑的 LaTeX 源码。我们通过适配器封装外部能力，并在依赖不可用时提供明确错误或可控降级。

项目源码及相关实现文件已发布在 GitHub 仓库：<https://github.com/YiXian0922Xue/software-architecture-course-design>。仓库包含应用代码、环境配置和启动脚本，可用于复现本文介绍的体系结构与核心流程。

1.2 课程设计目标

我们的设计目标包括：

- (1) **内容有据**：生成内容来自上传材料、OCR 结果或用户要求，并通过 RAG 提供证据。
- (2) **能力可替换**：OCR、嵌入、LLM、存储和导出能力均由独立服务封装。
- (3) **Demo 可运行**：我们采用 FastAPI、SQLite 和文件系统等轻量方案，并为嵌入服务提供本地降级。

2 用户需求分析

2.1 应用场景与目标用户

系统面向需要完成实验报告、课程设计报告或论文复现实验记录的高校学生。典型使用者在实验结束后手中拥有若干材料：教师给出的实验要求、实验报告模板、程序运行截图、实验结果图片、命令行输出、数据表格和少量个人记录。用户希望系统能够帮助其**快速整理这些材料，生成结构完整、图文对应、可继续编辑的报告源文件**。

本系统的利益相关者包括学生用户、教师/助教、开发维护者和系统管理员。学生用户关心上传是否方便、回答是否有依据、报告格式是否正确；教师和助教关心报告是否规范、是否虚构数据、是否能体现实验过程；开发维护者关心模块是否低耦合、接口是否清晰、外部服务是否容易替换；系统管理员关心本地部署是否简单、API 密钥是否安全、运行日志是否可诊断。

表 1: 系统利益相关者与关注点

角色	主要关注点	体系结构响应
学生用户	上传材料、检索材料、生成报告、下载 LaTeX 源码	单页 Web 工作区、后台任务、报告历史、可下载源码
教师/助教	报告结构规范、数据真实、过程可演示	RAG 证据约束、OCR 结果记录、图片插入可追踪、Demo 流程清晰
开发维护者	模块化、可测试、可扩展、易定位错误	三层 B/S、服务类隔离、仓储封装、适配器封装外部能力
系统管理员	本地部署、端口管理、密钥管理、日志查看	Conda 环境、start/stop 脚本、.env 配置、日志文件

2.2 功能性需求

根据项目目标，系统功能性需求如表2所示。需求编号用于后续功能分解、场景分析和测试验证。

表 2: 功能性需求表

编号	需求名称	详细说明与验收标准
FR-01	实验项目管理	用户可以新建项目、查看项目列表并切换项目。不同项目的材料、对话和报告互相隔离。
FR-02	多格式材料读取	支持 <code>.docx</code> 、 <code>.tex</code> 、 <code>.pdf</code> 、 <code>.txt</code> 、 <code>.md</code> 等文件的文本提取。
FR-03	模板识别与复用	用户可以将 LaTeX 文件标记为报告模板,系统在导出时尽量保留模板结构、宏包和章节组织。
FR-04	多图片 OCR	用户一次上传多张 <code>.png</code> 或 <code>.jpg</code> 截图,系统调用 OCR 提取图片中的实验结果文字。
FR-05	RAG 索引与问答	系统对材料切块、向量化、持久化,并在问答时返回与问题相关的材料片段及来源信息。
FR-06	报告智能生成	系统根据材料、模板、图片安排和用户要求生成结构化报告内容,避免无依据地编造实验数据。
FR-07	图片插入与约束	每张图片绑定稳定 ID、原文件名、尺寸和 OCR 文本;导出时每张图片恰好出现一次或进入兜底章节。
FR-08	LaTeX 源码导出	输出可继续编辑的 <code>.tex</code> 文件,并生成配套图片资源目录,便于用户继续编译与修改。
FR-09	历史记录	保存对话消息和报告记录,刷新页面后仍能查看历史生成结果。
FR-10	依赖状态提示	前端可查看 DeepSeek、OCR 和 Ollama 等依赖的配置与在线状态,便于定位演示故障。

2.3 非功能性需求

非功能性需求体现系统在生命周期中表现出的质量属性。本项目重点关注**易用性、响应性、可扩展性、可靠性、可维护性、安全性和可移植性**。

表 3: 非功能性需求表

质量属性	需求描述	设计响应
易用性	用户无需学习复杂命令，主要操作集中在页面完成。	使用 ChatGPT 风格单页界面，上传、问答、生成和下载集中在同一工作区。
响应性	本地查询和页面切换应快速完成，耗时 AI 调用要有进度提示。	报告生成采用后台任务，页面轮询状态并显示阶段信息。
可扩展性	未来可以替换 LLM、OCR、嵌入模型或向量库。	外部能力通过独立服务类和适配器封装。
可靠性	外部 API 失败不应生成虚假结果，上传文件和输出报告应可恢复。	错误显式返回；项目隔离目录；报告记录持久化。
可维护性	代码结构清晰，模块职责单一，依赖方向明确。	表示层、接口层、业务层、仓储层和基础设施层分层组织。
安全性	防止路径穿越、密钥泄漏和超大文件影响服务。	使用 <code>Path.name</code> 处理文件名， <code>.env</code> 不入库，限制文件大小。
可移植性	能在普通学生电脑上演示，避免复杂服务器运维。	使用 Conda、FastAPI、SQLite 和本地文件系统。

2.4 核心交互流程

系统的核心交互流程如下：

- (1) 用户访问本地 Web 页面，新建实验项目并填写实验名称。
- (2) 用户上传实验指导书、报告模板、运行日志和多张截图。
- (3) 系统根据文件类型执行**文档解析或 OCR**，将文本归一化后写入项目知识库。

- (4) 用户在对话框中询问材料相关问题，系统检索相关片段并调用 DeepSeek 基于证据回答。
- (5) 用户填写报告生成要求、图片安排和高级提示词，创建报告生成任务。
- (6) 报告智能体检索上下文，抽取模板章节，组织图片清单，调用模型生成结构化报告计划。
- (7) 导出器把计划转换为 LaTeX 章节、图表和图片资源目录，保存报告记录。
- (8) 用户下载 .tex 源码和图片资源，继续人工核验与编译。

3 功能分解与总体结构

3.1 顶层功能分解

按照功能模型思想，可以把系统看成由一组功能构件按层次组成，下层向上层提供服务。LabScribe Agent 的顶层功能可以分为项目与会话管理、材料摄取、RAG 知识检索、智能报告生成、LaTeX 导出和系统状态管理六部分。

表 4: 顶层功能模块分解

功能模块	子功能	输入/输出
项目与会话管理	项目创建、项目切换、对话历史、报告历史	项目信息、消息、报告记录
材料摄取	多文件上传、文档解析、模板识别、图片 OCR	文件路径到标准化文本
RAG 知识检索	文本切块、向量嵌入、余弦检索、来源返回	查询到相关 Chunk 列表
智能报告生成	上下文装配、图片规划、结构化 JSON 计划、错误处理	项目材料到报告中间表示
LaTeX 导出	模板替换、章节生成、figure 插入、资源目录生成	中间表示到 .tex 文件
系统状态管理	依赖检测、任务状态、日志提示、下载接口	状态 JSON 或文件响应

3.2 主要模块职责

从代码组织看，系统以 `app/` 目录为主体，分为接口层、领域层、服务层、仓储层和静态资源层。每个模块对应一个或多个体系结构构件，其职责如表5所示。

表 5: 模块职责与接口

模块	核心职责	典型输入/输出
<code>main.py</code>	创建 FastAPI 应用、挂载静态文件、注册路由。	HTTP 请求到应用入口。
<code>routes.py</code>	参数校验、HTTP 状态映射、项目、资源、聊天和报告接口。	JSON/Multipart 到 JSON/File。
<code>models.py</code>	定义 Project、Resource、ChatRequest、ReportRequest、Figure-Placement 等领域对象和请求对象。	Pydantic 数据模型。
<code>DocumentService</code>	解析 docx、pdf、tex、txt、md 等材料，将不同格式归一化为文本。	文件路径到文本。
<code>OCRService</code>	调用百度 OCR SDK 提取截图文字。	图片路径到 OCR 文本。
<code>EmbeddingService</code>	调用 Ollama 生成向量；不可用时使用确定性本地哈希嵌入。	文本列表到向量列表。
<code>RAGService</code>	切块、索引、相似度检索，并返回来源信息。	查询到 Top-K 片段。
<code>DeepSeekService</code>	封装 DeepSeek V4-Pro 调用、超时和错误转换。	Messages 到结构化回答。
ReportAgent	编排感知、检索、规划和导出流程，是智能体 workflow 核心。	Project 到 ReportRecord。
ReportService	将结构化报告计划转换为 XeLaTeX 源码和图片资源目录。	Markdown/计划到 TEX 文件。
<code>SQLiteRepository</code>	持久化项目、资源、块、消息和报告。	Domain Object 与 SQLite 互转。

模块	核心职责	典型输入/输出
app/static	前端页面、样式和交互逻辑。	用户操作到 HTTP 调用。

3.3 代码组织结构

项目代码结构符合**分层思想**:前端静态文件放在 `app/static`,HTTP 控制器放在 `app/api`,领域模型放在 `app/domain`,业务能力放在 `app/services`,持久化访问放在 `app/repositories`。这种组织方式能够让开发者快速判断代码变更影响范围,也便于在 4+1 视图的开发视图中呈现模块依赖。

```
software-architecture-course-design
|-- app
|   |-- api
|   |   `-- routes.py
|   |-- domain
|   |   `-- models.py
|   |-- repositories
|   |   `-- sqlite_repository.py
|   |-- services
|   |   |-- document_service.py
|   |   |-- embedding_service.py
|   |   |-- llm_service.py
|   |   |-- ocr_service.py
|   |   |-- orchestrator.py
|   |   |-- rag_service.py
|   |   `-- report_service.py
|   |-- static
|   |   |-- index.html
|   |   |-- app.js
|   |   `-- styles.css
|   `-- main.py
```

```
|-- BaiduAi
|-- data
|-- docs
|-- tests
|-- environment.yml
|-- requirements.txt
|-- run.py
|-- start.ps1
\-- stop.ps1
```

3.4 核心架构约束

为了避免智能体项目在功能增长后形成难以维护的“大脚本”，本系统设置如下依赖约束：

- (1) 表示层只通过 HTTP 调用接口层，不直接访问数据库和文件系统。
- (2) API 路由层负责协议转换和参数校验，不直接进行复杂业务决策。
- (3) 业务服务层可以依赖领域模型、仓储接口和外部适配器，但服务之间应保持单一职责。
- (4) 仓储层只负责持久化，不反向调用业务服务或前端逻辑。
- (5) 外部能力必须通过适配器进入系统，避免 DeepSeek、百度 OCR 或 Ollama 的调用细节扩散到各个模块。

这种约束体现了体系结构中“构件—连接件—配置”的组织关系：构件职责明确，连接件类型受控，配置可被视图描述和评估。

4 软件体系结构风格设计

软件体系结构风格是某一类系统组织方式的惯用模式。本项目不是单一风格，而是组合使用三层 B/S 风格、仓储风格、管道—过滤器风格和面向对象风格。不同风格作用于不同问题：三层 B/S 负责总体职责划分，仓储风格负责数据共享与一致性，管道—过滤器负责材料处理和生成流程，面向对象风格负责领域对象和服务对象的封装。

4.1 三层 B/S 风格

系统总体采用三层 B/S 风格，即浏览器表示层、FastAPI 业务层、SQLite/文件系统数据层。该风格与课程中三层 C/S、B/S 的思想一致：表示层负责界面交互，功能层负责业务逻辑，数据层负责数据管理。

表 6: 三层 B/S 风格在系统中的映射

层次	系统构件	职责
表示层	index.html、app.js、styles.css	提供项目创建、文件上传、RAG 问答、报告生成和下载入口。
业务层	FastAPI、API Routes、ReportAgent、Services	完成参数校验、任务编排、材料处理、检索、模型调用和导出。
数据层	SQLite、data/、上传目录、输出目录	保存项目元数据、文本块、向量、消息、报告和图片资源。

三层 B/S 风格的优点是用户端无需安装专门客户端，浏览器即可使用；后端集中管理 AI 能力和文件访问，便于维护；前后端通过 HTTP 接口隔离，未来也可以扩展为移动端或命令行客户端。缺点是所有 AI 任务都经过后端服务，当外部 API 响应慢时，后端需要更细致的任务调度和超时控制。当前课程 Demo 通过后台任务和前端轮询缓解等待问题。

4.2 仓储风格

仓储风格把共享数据放在统一仓储中，多个构件通过仓储读写数据。在本系统中，SQLite 与工作区文件系统共同构成轻量仓储：SQLite 保存项目、资源、文本块、向量、消息和报告记录；文件系统保存原始上传材料、生成的 .tex 文件和图片资源目录。

采用 SQLite 的原因是课程 Demo 不需要部署复杂数据库，SQLite 具备事务能力、备份简单、跨平台可用，适合个人项目和本地演示。向量以 JSON 形式保存在数据库中，检索时在内存计算余弦相似度。该实现对课程规模足够简单可靠；如果将来材料量扩大，可以把仓储层替换为 FAISS、Chroma 或 pgvector，而不必改变上层 API。

仓储风格的风险在于数据层可能成为性能瓶颈或耦合中心。因此系统限制业务服务只通过 SQLiteRepository 访问数据，避免多个模块直接拼 SQL；同时把原始文件放在文件系统而不是数据库 BLOB 中，减少数据库膨胀。

4.3 管道—过滤器风格

材料摄取与报告生成天然具有流水线特征，适合使用**管道—过滤器风格**。每个过滤器只负责一种转换，前一阶段输出作为后一阶段输入。

表 7: 管道—过滤器风格的流程映射

流程	过滤器序列
材料摄取	上传文件 → 类型判断 → 文档解析/OCR → 文本归一化 → 切块 → 嵌入 → 持久化
RAG 问答	用户问题 → 查询嵌入 → 相似度检索 → 上下文装配 → LLM 回答 → 引用展示
报告生成	写作要求 → 材料检索 → 模板章节提取 → 结构化计划 → LaTeX 导出 → 下载

该风格提升了系统可测试性。例如，文本切块、OCR、嵌入、检索、导出可以分别进行单元测试。缺点是如果阶段过多，调试时需要查看中间状态；因此系统在前端显示任务阶段，在后端打印日志，帮助定位是解析、检索、LLM 还是导出阶段出错。

4.4 面向对象风格

在逻辑视图中，系统以 Project、Resource、Chunk、Message、ReportRecord 等领域对象为中心，同时通过 DocumentService、OCRService、RAGService、ReportAgent 等服务对象封装行为。这种面向对象风格强调**抽象、封装和组合**：领域对象保持数据结构一致，服务对象承担特定职责，外部能力通过适配器对象接入。

本项目没有大量使用继承，而是**更多使用组合关系**。例如，ReportAgent 组合使用 RAGService、DeepSeekService 和 ReportService；RAGService 组合使用 EmbeddingService 和 SQLiteRepository。这样能降低继承层次带来的复杂性，也符合智能体系统中“能力插件化”的需求。

4.5 风格选择总结

表 8: 体系结构风格适配性分析

风格	解决的问题	优点	风险与控制
三层 B/S	前端、业务、数据职责划分	易部署、易演示、接口清晰	长任务阻塞；通过后台任务和轮询缓解
仓储风格	多构件共享项目数据和向量	数据一致、免运维、易备份	线性扫描扩展性有限；可替换向量库
管道—过滤器	材料处理和报告生成流程	阶段清晰、可测试、可替换	中间状态多；通过日志和任务阶段可观测
面向对象风格	领域对象和服务职责组织	封装良好、低耦合、便于复用	过度抽象风险；保持服务单一职责

5 智能体设计模式分析

智能体系统的核心不只是“调用大模型”，而是围绕感知、记忆、推理、工具使用、执行和反馈形成稳定 workflow。本系统结合了 RAG、工具使用、提示链/规划、路由、记忆管理、护栏和异常恢复等智能体设计模式。

5.1 RAG：检索增强生成模式

RAG 模式用于解决大模型缺少用户私有材料和容易产生幻觉的问题。系统先将用户上传材料切块并向量化，在问答或报告生成前检索相关片段，将片段内容、来源文件和相似度作为上下文提供给 LLM。这样，模型不是凭空写报告，而是在给定证据范围内组织语言。

在本系统中，RAG 的执行步骤如下：

- (1) DocumentService 和 OCRService 将异构材料转换为统一文本。
- (2) RAGService 将文本切分为带资源来源的 Chunk。
- (3) EmbeddingService 为 Chunk 生成向量并保存到 SQLite。
- (4) 用户提问或生成报告时，再为查询生成向量。

(5) 系统计算查询向量与项目内各 Chunk 的余弦相似度，选取 Top-K 片段。

(6) DeepSeekService 将检索片段与用户任务合并到提示中，生成回答或报告计划。

RAG 模式的优势是提高事实可追踪性，减少模型凭空编造实验过程。局限是检索质量取决于分块策略、嵌入模型和 Top-K 设置；如果原材料中没有某个实验结果，系统仍可能无法回答。因此系统提示要求模型在证据不足时说明，而不是补写不存在的数据。

5.2 Tool Use: 工具使用模式

实验报告生成涉及大量模型不擅长直接完成的确定性操作，例如读取 docx、解析 PDF、识别图片文字、复制图片、生成 LaTeX 路径和写文件。工具使用模式把这些能力封装为独立工具，由智能体编排调用。

本系统中的主要工具包括：

- **文档解析工具**：将 docx、pdf、tex、txt、md 转换为纯文本。
- **OCR 工具**：调用百度 OCR 识别截图中的文字。
- **嵌入工具**：调用 Ollama 或本地哈希算法生成向量。
- **检索工具**：在 SQLite 中查找与问题最相关的 Chunk。
- **导出工具**：将结构化计划转换为 LaTeX 源码和图片目录。

采用工具模式后，大模型只负责语言理解、内容组织和计划生成，文件读写、路径合法性和 LaTeX 语法由本地工具确定性完成。这显著降低了模型出错后直接破坏文件系统或生成不可控路径的风险。

5.3 Prompt Chaining 与 Planner-Executor 模式

复杂报告生成若由单个长提示一次完成，容易出现章节遗漏、图片重复、模板被破坏或结果格式不稳定。因此系统使用**提示链与计划—执行思想**，把任务拆分为多个阶段：**模板分析、材料检索、图片清单组织、结构化 JSON 计划生成、计划校验、LaTeX 导出**。

ReportAgent 在该模式中扮演 **Planner**：它根据项目材料和用户要求组织提示，让 DeepSeek 返回可验证的结构化计划。**ReportService** 扮演 **Executor**：它不让模型直接写文件，而是根据计划执行章节生成、图片插入和资源复制。这样即使模型输出存在局部问题，系统也可以通过 JSON 解析、图片 ID 检查和兜底章节修复来减少错误扩散。

5.4 Routing: 路由模式

系统需要根据文件类型、资源种类和任务类型选择不同流程，这体现了路由模式。文件上传时，系统根据扩展名判断进入文档解析、OCR 或模板处理流程；用户请求时，根据是聊天问答还是报告生成进入不同服务；依赖状态检测时，根据 Ollama 是否在线选择真实嵌入或本地哈希嵌入。

这种路由使系统从固定执行路径转变为上下文感知的动态流程。对用户而言，只需上传材料和输入需求；对后端而言，不同资源和任务会被分派给合适的工具或子流程。

5.5 Memory Management: 记忆管理模式

智能体需要保存项目材料、对话历史和生成报告记录，才能支持连续交互和历史追踪。本系统的记忆分为三类：

- (1) **短期记忆**：当前请求中的用户问题、写作要求和临时检索上下文。
- (2) **项目记忆**：已上传材料、OCR 文本、向量块和项目级历史消息。
- (3) **结果记忆**：已生成报告的路径、格式、时间和下载记录。

项目记忆和结果记忆保存在 SQLite 与文件系统中。刷新页面后，用户仍可继续查看历史消息和报告记录。这种记忆管理使系统不再是无状态问答，而是围绕实验项目持续积累上下文。

5.6 Guardrails 与异常恢复模式

由于本项目直接面向课程报告生成，如果模型虚构数据或外部服务失败后仍输出“看似正确”的内容，会严重影响报告可信度。因此系统使用护栏和异常恢复模式。

护栏包括：文件大小限制、文件名目录成分清理、上传资源类型校验、图片 ID 唯一性校验、DeepSeek JSON 结构校验、OCR 失败显式提示、禁止无依据编造实验数据等。**异常恢复包括**：Ollama 不可用时降级为本地哈希嵌入；DeepSeek 或百度 OCR 不可用时向前端返回可读错误；报告生成失败时记录任务状态和错误消息，避免用户长时间等待。

护栏模式的代价是系统代码复杂度增加，生成速度可能下降，但它换来了更强的可靠性、可诊断性和课程演示安全性。

6 4+1 视图分析

4+1 视图模型从多个利益相关者关心的角度描述软件体系结构。**逻辑视图面向最终用户的功能需求，开发视图面向程序员的软件管理，进程视图面向系统集成人员关注的性能、并发和通信，物理视图面向系统工程人员关注的拓扑、安装和部署，场景视图则用关键用例把四种视图联系起来。**逻辑视图和开发视图主要描述系统静态结构，进程视图和物理视图则更关注运行时和部署时结构；**场景视图是最重要的需求抽象之一，因为它能够检验各视图之间是否一致。**

本报告中的 4+1 视图图形严格参照课程课件的 Booch 风格和示意方式绘制：逻辑视图使用云状类/服务构件和关联、使用、聚集连接；开发视图采用 5 层子系统结构，并在左侧使用平行水平箭头表示不同层次来源，在右侧表示领域特定/领域无关、客户定制/通用代码之间的关系；进程视图使用扩展 Ada 任务样式表示进程和任务；物理视图使用处理器、设备和通信连接表示软件到硬件节点的映射；场景视图使用云状对象和编号箭头表示关键活动。

6.1 逻辑视图：核心类与组件结构

逻辑视图主要支持系统的功能需求，即系统对最终用户提供的服务。**LabScribe Agent 的核心对象模型围绕“实验项目”展开。Project 是聚合根，Resource、Chunk、Message 和 ReportRecord 都依附于某个 Project。服务类则围绕项目材料处理和报告生成协同工作：ApiController 接收用户请求，ReportAgent 编排智能体流程，DocumentService 与 OCRService 负责感知输入，RAGService 和 EmbeddingService 负责知识检索，DeepSeek-Service 负责语言生成，ReportService 负责输出执行，SQLiteRepository 负责持久化。**

逻辑视图

用 Booch 逻辑构件与连接件描述 LabScribe Agent 的核心对象模型和服务协作。

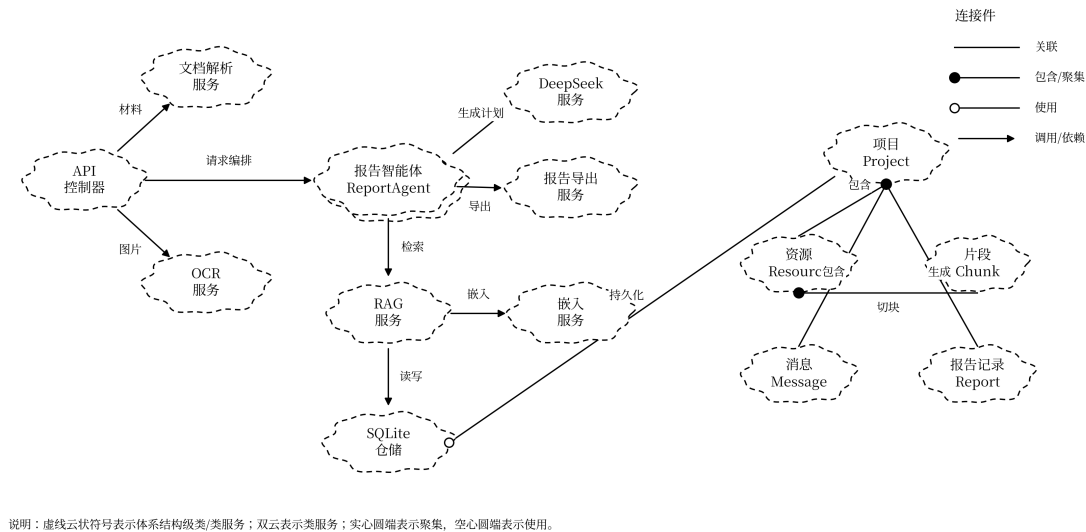


图 1: LabScribe Agent 逻辑视图

图1中，虚线云状构件表示体系结构层面的类或服务。**Project** 作为中心领域对象聚合 **Resource**、**Chunk**、**Message** 和 **ReportRecord**，体现实验项目中材料、知识块、对话和报告的统一边界。**ReportAgent** 连接 **RAGService**、**DeepSeekService** 和 **ReportService**，说明报告生成是由检索、推理和导出共同完成的复合行为。**RAGService** 连接 **EmbeddingService** 和 **SQLiteRepository**，体现知识检索既依赖向量能力，也依赖持久化数据。外部能力没有直接散落在业务流程中，而是由 **DeepSeekService**、**OCRSERVICE** 和 **EmbeddingService** 作为适配器接入。

该逻辑视图体现了面向对象风格中的封装与组合。系统尽量保持单一、内聚的对象模型：项目是用户操作边界，资源是输入边界，报告是输出边界，服务类以职责协作而非继承层次连接。这样既便于理解功能需求，也便于在后续修改中定位影响范围。

6.2 开发视图：模块划分与代码组织

开发视图也称模块视图，主要侧重软件模块的组织和管理。课程中强调开发视图最好采用 4-6 层子系统，并且每个子系统只与同层或更低层的子系统通信，以避免复杂依赖关系。LabScribe Agent 采用 5 层开发视图：表示与工具层、应用功能层、领域与编排层、支撑机制层、基础服务层。

开发视图

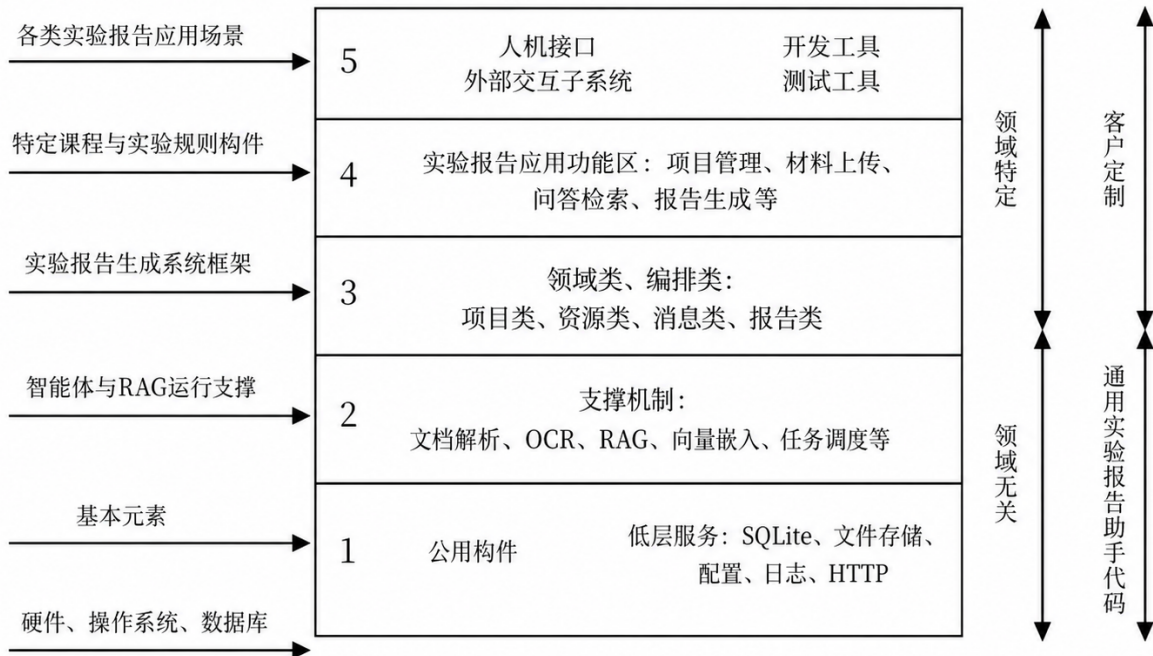


图 2: LabScribe Agent 开发视图

图2按照课程开发视图模板绘制。左侧平行箭头说明不同层次的输入来源：各类实验报告应用场景对应最上层的人机接口和外部交互；特定课程与实验规则构件对应应用功能区；实验报告生成系统框架对应领域类和编排类；智能体与 RAG 运行支撑对应文档解析、OCR、RAG、向量嵌入和任务调度；基本元素及硬件、操作系统、数据库支撑最底层公用构件和低层服务。

五层含义如下：

- (1) **第 5 层：人机接口与外部交互子系统。**对应 `app/static` 的前端页面，以及开发工具、测试工具和本地演示脚本。该层最接近用户需求，变化频率最高。
- (2) **第 4 层：实验报告应用功能区。**对应项目管理、材料上传、问答检索、报告生成、报告下载等业务功能，主要由 API Routes 和部分业务服务承载。
- (3) **第 3 层：领域类与编排类。**对应 Project、Resource、Message、ReportRecord 等领域对象，以及 ReportAgent 编排逻辑。该层把应用功能抽象为稳定的领域概念。

- (4) **第 2 层：支撑机制**。对应文档解析、OCR、RAG、向量嵌入、任务调度、LLM 调用和导出等可复用能力。它服务于上层应用功能，也可被未来其他实验类应用复用。
- (5) **第 1 层：公用构件与低层服务**。对应 SQLite、文件存储、配置、日志、HTTP 客户端和本地运行环境。该层领域无关、通用性强。

右侧关系说明系统代码从上到下逐渐由领域特定转向领域无关。上层是客户定制的实验报告助手功能，下层是通用实验报告助手代码与通用基础服务。**这样的层次结构保证了需求变化时修改主要集中在上层，而底层基础机制可以稳定复用。**

6.3 进程视图：并发、调度与通信机制

进程视图侧重系统运行时特性，关注并发性、分布性、通信机制、容错能力以及逻辑视图中的主要抽象如何映射到运行时任务。LabScribe Agent 的运行时参与者包括浏览器前端进程、FastAPI/Uvicorn 服务进程、报告生成后台任务、Ollama 本地服务、DeepSeek 云端 API、百度 OCR 云端 API、SQLite 和文件系统访问任务。

进程视图

用扩展 Booch/Ada 任务符号表示运行时进程、后台任务、远程调用和双向消息。

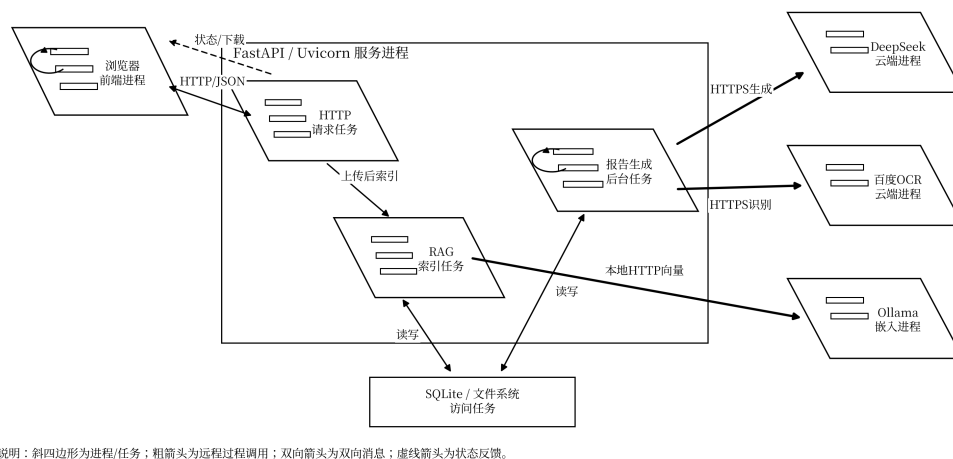


图 3: LabScribe Agent 进程视图

图3使用扩展 Booch/Ada 任务风格表示进程与任务。浏览器前端进程负责用户交互、文件上传和任务状态轮询；FastAPI/Uvicorn 服务进程包含 HTTP 请求处理任务、材料解析与索引

任务、报告生成后台任务和下载响应任务；Ollama 嵌入服务进程负责向量生成；DeepSeek 云端 LLM 进程负责结构化回答和报告计划；百度 OCR 云端进程负责图片文字识别；SQLite/文件系统任务负责持久化。

通信机制可分为以下几类：

- **前端与后端：**浏览器通过 HTTP/JSON/Multipart 与 FastAPI 通信，报告生成阶段通过轮询任务状态获取进度。
- **后端与本地模型：**FastAPI 通过本地 HTTP 调用 Ollama 的嵌入接口；若 Ollama 不在线，则在进程内执行哈希嵌入降级。
- **后端与云服务：**FastAPI 通过 HTTPS 调用 DeepSeek 和百度 OCR，属于远程过程调用。
- **后端与本地存储：**通过 SQL 和文件读写访问 SQLite、上传目录和输出目录。

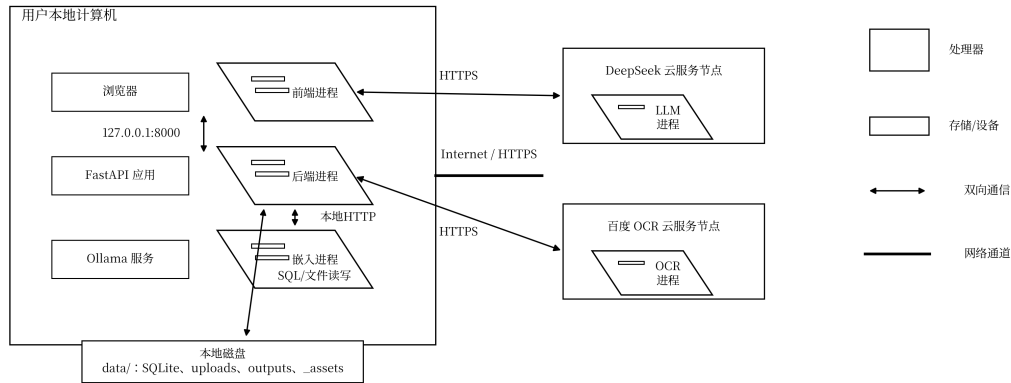
当前 Demo 使用单个 FastAPI 进程即可完成演示，但报告生成是长任务，**系统使用后台任务和状态轮询避免前端阻塞**。若生产化部署，可将 OCR、嵌入和报告生成迁移到独立工作进程，并使用队列连接 API 进程与工作进程，以提高吞吐和容错能力。

6.4 物理视图：部署结构与软硬件映射

物理视图主要考虑如何把软件映射到硬件节点上，并解决系统拓扑、安装、通信、性能和可靠性问题。LabScribe Agent 是一个本地单机应用加外部云 API 的混合部署系统：主体软件部署在用户本地计算机，DeepSeek 和百度 OCR 部署在外部云服务节点，Ollama 可选部署在本机。

物理视图

系统主体映射到用户本地计算机，LLM 与 OCR 能力映射到外部云服务节点。



说明：大矩形为处理器节点，小矩形为设备或部署构件，箭头表示节点通信或本地读写。

图 4: LabScribe Agent 物理视图

图4中，本地计算机节点包含浏览器、Python/Conda 环境、FastAPI/Uvicorn 服务、LabScribe Agent 应用代码、可选 Ollama 服务和本地磁盘。浏览器与 FastAPI 通过 127.0.0.1:8000 通信；FastAPI 与 Ollama 通过 127.0.0.1:11434 通信；FastAPI 与本地磁盘通过 SQL 和文件读写交互。外部节点包括 DeepSeek 云服务节点和百度 OCR 云服务节点，它们通过 HTTPS 与后端通信。

该部署结构的优势是演示成本低，用户只需一台普通电脑即可运行，不需要 Kubernetes、云服务器或数据库运维。风险是本地资源有限，云 API 依赖网络和密钥配置。为降低风险，系统支持 Ollama 离线降级，对 OCR 和 DeepSeek 错误进行显式提示，并将数据保存在本地可备份目录中。

6.5 场景视图：关键用例与流程串联

场景视图是 4+1 中的“+1”，用于把逻辑视图、开发视图、进程视图和物理视图有机联系起来。本项目最关键的场景是“学生上传实验材料并生成 LaTeX 实验报告”。该场景覆盖项目管理、材料上传、OCR、RAG、LLM、LaTeX 导出、文件下载等核心构件，也能体现智能体的感知、决策和执行。

场景视图

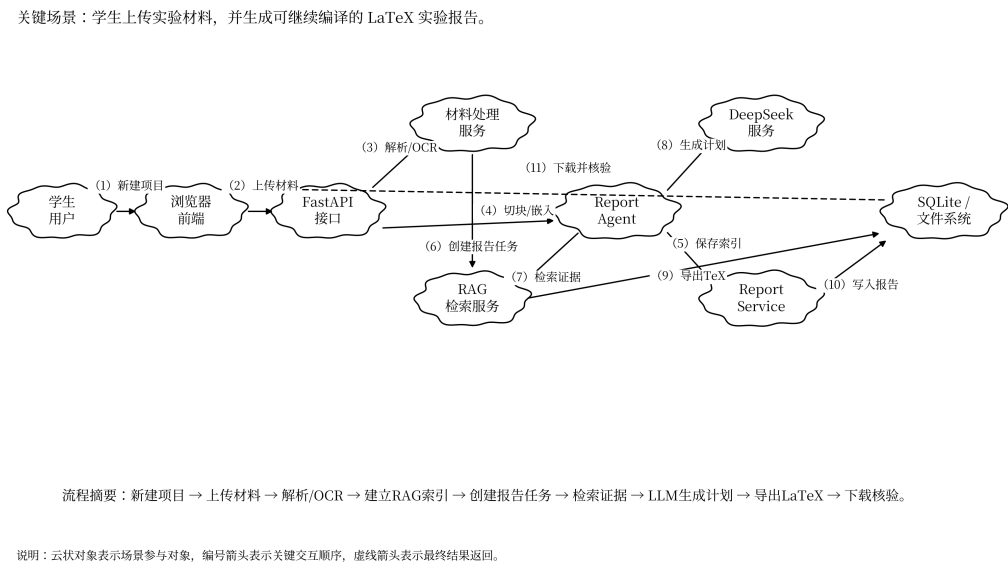


图 5: LabScribe Agent 场景视图：实验报告生成场景

图5中，各对象以云状构件表示，箭头编号表示交互顺序。**场景从学生用户新建项目开始，经过浏览器前端、FastAPI 接口、材料处理服务、RAG 检索服务、ReportAgent、DeepSeek 服务、ReportService 和 SQLite/文件系统，最终回到用户下载报告。**该场景能够验证前文各视图的一致性：逻辑视图中的核心服务都参与流程；开发视图中的层次依赖按从上到下方向发生；进程视图中的浏览器、后端、本地服务和云服务均被调用；物理视图中的本地节点和外部节点也都发生通信。

6.6 五个视图之间的一致性

4+1 视图不是五张孤立的图，而是对同一体系结构的不同切面描述。LabScribe Agent 中五个视图的一致性可以从表9看出。

表 9: 4+1 视图一致性分析

视图	关注问题	与其他视图的关系
逻辑视图	系统有哪些核心类和服务，如何支持功能需求。	服务类在开发视图中映射为模块，在进程视图中映射为任务或调用。

视图	关注问题	与其他视图的关系
开发视图	代码如何分层组织，模块如何依赖。	层次结构约束逻辑构件实现位置，也影响部署包和测试范围。
进程视图	系统运行时如何并发、调度和通信。	逻辑服务的调用在运行时表现为本地函数、HTTP、HTTPS、SQL 或文件读写。
物理视图	软件部署在哪些硬件和外部服务节点上。	进程视图中的运行单元被映射到本地计算机、Ollama、本地磁盘和云服务。
场景视图	关键用例如何串联全部构件。	检验四个视图是否能够共同支持“上传材料并生成报告”的核心需求。

7 数据设计与接口设计

7.1 数据模型设计

我们以 Project 为聚合边界组织持久化数据，并将文件保存在工作区中，数据库只记录元数据、解析结果、向量和路径。

表 10: 核心数据表设计

数据表/对象	关键字段	说明
projects	标识、标题、创建时间	项目元数据与聚合边界。
resources	项目标识、类型、路径、解析文本	上传材料、模板和图片。
chunks	资源标识、内容、向量	用于 RAG 检索的文本块。
messages	项目标识、角色、内容	项目内的问答历史。
reports	项目标识、格式、路径	报告记录与下载位置。

我们通过统一仓储对象访问这些数据，避免业务服务重复实现数据操作。

7.2 API 接口设计

我们使用 FastAPI 提供 REST 接口，并按项目、资源、问答和报告任务划分职责。

表 11: 核心 API 摘要

接口	功能	说明
GET /api/health	状态检测	返回外部依赖状态。
POST /api/projects	创建项目	建立报告项目。
POST /api/projects/{id}/resources	上传材料	解析并建立索引。
POST /api/projects/{id}/chat	RAG 问答	检索材料并回答。
POST /api/projects/{id}/report-jobs	创建任务	后台生成报告。
GET /api/report-jobs/{job_id}	查询任务	返回阶段与结果。
GET /api/reports/{id}/download	下载报告	返回生成的源码。

7.3 RAG 检索算法

我们将材料切分为带少量重叠的文本块，生成向量后写入 SQLite。查询时仅在当前项目中计算余弦相似度：

$$\text{sim}(q, d) = \frac{q \cdot d}{\|q\| \|d\|}$$

其中 q 为查询向量， d 为文本块向量。我们返回相似度最高的 Top-K 片段及其来源；课程规模下采用线性扫描即可，后续可替换为专用向量索引。

8 关键实现机制

我们采用“本地资源管理、OCR 理解、模型规划、导出器执行”的图片处理流程。上传后为图片保存稳定标识和 OCR 文本，模型只规划章节与图注，ReportService 负责复制文件并生成唯一的 **figure** 引用。

对于 LaTeX 模板，我们优先替换预设占位符；模板无占位符时，在 `\end{document}` 前插入正文；未提供模板时生成基础 **ctexart** 文档。报告生成使用后台任务，前端依据 job ID 轮询阶段与结果，避免长请求阻塞。

我们对关键异常采用显式处理和有限降级：

- Ollama 不在线时使用本地哈希嵌入；
- OCR 或 DeepSeek 失败时返回明确错误，不伪造结果；
- 对不支持或过大的文件拒绝处理，同时保留其他资源的处理结果。

9 体系结构质量评估

我们借鉴 SAAM/ATAM，以代表性场景检查体系结构对主要质量属性的支持。

9.1 质量属性场景

表12列出主要质量场景。

表 12: 质量属性场景分析

质量属性	刺激源/刺激	环境与制品	期望反应	反应度量
可靠性	OCR/LLM 调用失败	外部网络不稳定	返回错误且不伪造报告	错误可见、无虚假记录
可维护性	替换 OCR 服务	系统正常运行	仅替换服务适配器	其他模块无须修改
可扩展性	文本块显著增长	线性检索变慢	替换向量存储	上层接口保持不变

质量属性	刺激源/刺激	环境与制品	期望反应	反应度量
安全性	上传恶意文件名	本地运行	清理路径并隔离保存	不发生目录穿越
响应性	创建生成任务	模型调用耗时较长	后台执行并显示状态	页面不阻塞

9.2 敏感点与权衡点

我们识别出的敏感点包括向量存储、外部 API、模板插入和内存任务状态。主要权衡为：

- SQLite 便于部署，但限制大规模向量检索性能；
- 外部模型提升生成与图片理解质量，但增加延迟和可用性风险；
- 后台任务改善交互体验，但需要维护状态与错误传播。

当前体系结构可直接支持材料上传、RAG 问答和 LaTeX 生成；Word/PDF 导出与多人协作需要新增服务；超大材料库则要求引入向量数据库。这些场景明确了系统边界。

10 Demo 实现与运行说明

10.1 实验环境

我们主要在 Windows 环境演示，后端使用 Python、FastAPI 和 Uvicorn，前端使用原生 HTML/CSS/JavaScript，数据保存在 SQLite 与本地文件系统中。系统接入百度 OCR、DeepSeek 和可选的 Ollama，输出 XeLaTeX 源码及图片资源。

10.2 安装与启动

首次安装执行：

```
conda env create -p .conda\env -f environment.yml
ollama pull nomic-embed-text
```

启动与关闭执行：

```
.\start.ps1 # 后台启动，访问 http://127.0.0.1:8000
.\stop.ps1  # 安全关闭并释放 8000 端口
```

我们避免同时以前台和后台方式运行服务，以免占用同一端口；提交代码前还需确认 `.env` 已被忽略并及时轮换密钥。

10.3 演示流程设计

我们的演示流程如下：

- (1) 新建项目并上传实验材料、模板和截图；
- (2) 展示 OCR/RAG 索引，并完成一次带来源的材料问答；
- (3) 创建报告任务，观察任务状态；
- (4) 下载并检查 `.tex` 与图片资源。

11 结论

本次课程作业中，我们围绕高校实验报告编写场景，完成了 **LabScribe Agent 的需求分析、功能分解、体系结构设计、4+1 视图建模与可运行 Demo**。系统以三层 B/S 为总体结构，结合仓储和管道—过滤器风格，将材料解析、OCR、向量检索、模型调用与 LaTeX 导出组织为边界清晰的构件。通过 **RAG、工具使用、路由、记忆管理和异常恢复等智能体模式**，我们使报告生成过程能够利用用户材料，并降低无依据生成和外部服务故障带来的影响。

4+1 视图从不同角度验证了设计与实现的一致性：逻辑视图说明核心服务之间的协作关系，开发视图对应代码模块和依赖边界，进程视图描述浏览器、后端与外部服务之间的通信，物理视图给出本地应用与云端能力的部署映射，场景视图则贯穿材料上传、检索问答和报告生成的完整流程。结合质量场景和 Demo 验证结果，我们认为**当前结构能够满足课程规模下的功能、可维护性、可移植性和可靠性要求**。

当前实现仍存在一定边界，例如 **SQLite 线性向量检索不适合大规模材料库**，**OCR 与大模型能力依赖外部服务**，**内存任务状态也难以支持多实例部署**。但这些限制已被隔离在存

储、适配器和任务机制中，后续可以通过**替换向量数据库、引入持久化任务队列和完善模板语义协议**逐步改进。总体而言，我们不仅实现了一个可演示的智能体应用，也形成了一套**可解释、可验证且便于演进的软件体系结构**，为继续扩展实验知识管理与辅助写作能力提供了基础。

附录：个人分工表

姓名	学号	负责模块	承担权重
薛以贤	2312386	完成前端页面与交互逻辑开发，实现 LaTeX 导出、图片插入与前后端联调；撰写需求分析、4+1 视图、体系结构风格并进行文档排版整理。	50%
李宇帆	2312391	完成 FastAPI 后端接口、SQLite 持久化、材料解析、OCR、Ollama 嵌入、RAG 检索与 DeepSeek 调用调试；撰写功能分解、智能体模式、接口设计和运行说明。	50%